

PilotFish Data Attribute Swapper Processor – Complete Reference (26R1.11 WAR)

Derived from decompiled eip.war.hs.26R1.11

July 24, 2026

At a glance (plain English)

Think of this as a **save** / **load** / **swap** step for the current message. The data flowing through a transaction can be copied into a named “parking spot” (an attribute), copied back out, or exchanged—so later steps can hide, restore, or reuse that content without losing it.

Data Attribute Swapper — what it does

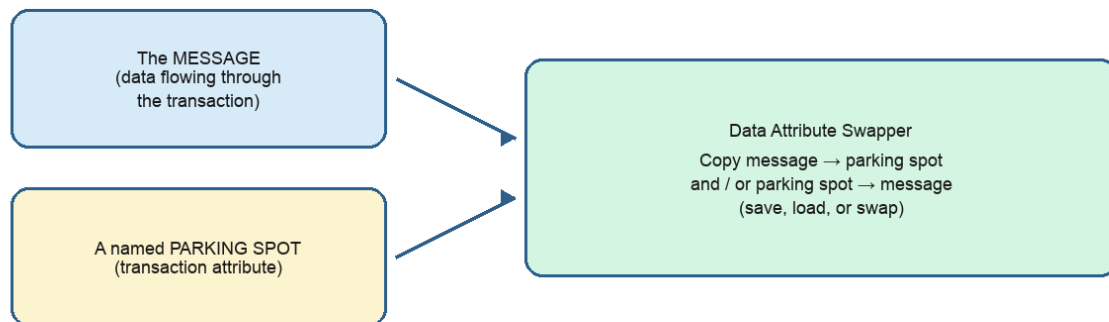


Figure 1: At a glance (plain English)

- **Save** the current message into a named label.
- **Load** a previously saved label back into the message.
- **Swap** the two in one step when both directions are enabled.
- Optional modes for text, XML documents, or large/binary streams; can also use global shared labels.

Purpose

This document is a **deep dive** into the **Data Attribute Swapper** processor as shipped in **eip.war.hs.26R1.11**: how it copies transaction **data streams** into **attributes** (and the reverse), storage modes (String / DOM / Stream), global attribute buckets, configuration options, and failure modes.

Provenance	Value
WAR	<code>eip.war.hs.26R1.11</code>
Module JAR	<code>xcs-modules-core-1.0.3.jar</code>
Core JAR	<code>xcs-core-1.2.6.jar</code>
Decompiler	CFR 0.152
Implementation class	<code>com.pilotfish.eip.modules.internal.SaveData</code>
Decompiled path	<code>war-pipeline/decompiled/eip-26R1.11/.../Sa</code>
Processor type string	Data Attribute Swapper
Description	Swaps the current transaction data into or out of an attribute
Help ID	<code>console.processor.attributeswapper</code>
Categories	ATTRIBUTES, PILOT_FISH

Related APIs:

Component	Role
<code>TransactionData</code>	Holds the mutable data stream
<code>TransactionAttributes</code>	Per-transaction attribute map
<code>TransactionAttributes</code> global APIs	Cross-transaction buckets
<code>EIPCacheManager</code> / <code>Cache</code>	Buffering streams / DOM serialization
<code>AbstractProcessor</code>	Base processor configuration

Derived from WAR bytecode (CFR). Narrative follows executable logic in `processData`.

1. High-level behavior

The processor does **not** transform message formats. It **moves or copies** the current transaction payload between:

- the **transaction data stream** (`TransactionData.getDataStream()` / `setDataStream()`), and
- a **named attribute** (transaction-local or **global** bucket).

Direction is controlled by two independent booleans (they can both be true = true swap):

Flag	UI label	Tag	Default
Data -> Attribute	Copy Data to Attribute	SAVEINTAG	true
Attribute -> Data	Copy Attribute to Data	SAVEOUTTAG	false

Storage representation is controlled by mutually exclusive modes (descriptor tooltips say so; code prefers DOM if both were somehow true because `isDom` is checked first):

Mode	UI label	Tag	Default	Attribute value type
Plain (neither)	–	–	default path	String
DOM	Use Document Object Model	USEDOCUMENTOBJECTMODEL	DOM	<code>org.w3c.dom.Document</code>
Stream	Use Stream	StoreAsStream	stream	<code>InputStream</code> (from EIP cache)

Entry point: `processData(TransactionData data)` -> returns the same `TransactionData` instance with stream and/or attributes updated.

2. Configuration reference

Items from `SaveDataToAttributeProcessor.getConfigurationDescriptor()` (plus super).

2.1 Attribute Name

Property	Value
UI label	Attribute Name
Tag	<code>AttributeName</code>
Type	String (required)
Default	<code>com.pilotfish.eip.modules.SaveDataToAttributeProcessor</code>
Tooltip	Specifies the attribute to operate upon

Nuances:

- Use reverse-DNS style names in real routes (e.g. `com.acme.SavedPayload`) to avoid collisions.
- The default is a placeholder name; leave it only if that exact key is intentional.
- For global mode, this is the key **inside** the chosen bucket.

2.2 Copy Attribute to Data

Property	Value
UI label	Copy Attribute to Data
Tag	SAVEOUTTAG
Type	Boolean
Default	false
Tooltip	Copies attribute contents to transaction data. IMPORTANT: if the attribute does not already exist, an exception is thrown

Nuances:

- Missing / null attribute always fails via `buildErrorMessage(...)`.
- Global vs local changes only *where* the attribute is read from.

2.3 Copy Data to Attribute

Property	Value
UI label	Copy Data to Attribute
Tag	SAVEINTAG
Type	Boolean
Default	true
Tooltip	Copies transaction data contents to the attribute

Nuances:

- Default configuration alone = **save current stream into the attribute** (String mode) and leave the stream rewound from cache (see §4).
- Combined with Copy Attribute to Data = **swap**.

2.4 Use Document Object Model

Property	Value
UI label	Use Document Object Model
Tag	USEDOCUMENTOBJECTMODEL
Type	Boolean
Default	false
Tooltip	Store/retrieve via DOM for XSLT access. Mutually exclusive with Use Stream

Nuances:

- Parses XML with a pooled `DocumentBuilder` (`namespaceAware=true`).
- Attribute holds a live `Document` object (not a String).
- Useful when later XSLT / processors expect a DOM attribute rather than re-parsing text.

2.5 Use Stream

Property	Value
UI label	Use Stream
Tag	<code>StoreAsStream</code>
Type	Boolean
Default	<code>false</code>
Tooltip	Store/retrieve via cache <code>InputStreams</code> . Mutually exclusive with Use Document Object Model

Nuances:

- Avoids materializing the whole payload as a Java `String` (better for large binary / large text).
- Attribute value is an `InputStream` obtained from `Cache.getInputStream()`.
- Callers that later read the attribute must treat it as a stream (consume carefully; do not assume `String`).
- Send Email / attachment patterns in the product often recommend this mode when attributes hold file bytes.

2.6 Global Attribute

Property	Value
UI label	Global Attribute
Tag	<code>IsGlobalAttribute</code>
Type	Boolean
Default	<code>false</code>
Tooltip	Toggles if attribute is globally accessible to all transactions

Dependency: when true, shows **Global Attribute Bucket**.

Uses `TransactionAttributes.setGlobalAttribute(bucket, name, value)` / `getGlobalAttribute(...)` instead of the per-transaction map.

2.7 Global Attribute Bucket

Property	Value
UI label	Global Attribute Bucket
Tag	<code>GlobalAttributeBucket</code>
Type	String (required when visible)
Default	<code>DEFAULT</code>
Tooltip	Name of the bucket in which a globally accessible attribute resides

Nuances:

- Buckets partition global state across unrelated uses.
- Global attributes survive across transactions in the JVM process; they are **not** a durable database. Restart / redeploy clears them unless something else repopulates them.
- Concurrent routes writing the same bucket+name can race; design accordingly.

3. Direction combinations (conceptual)

Copy Data to Attribute	Copy Attribute to Data	Effect (typical)
true	false	Save: payload -> attribute; stream restored from cache copy (mode-dependent)
false	true	Load: attribute -> data stream (attribute must exist)
true	true	Swap: payload saved to attribute; previous attribute value becomes new stream
false	false	No-op for data movement (modes still evaluated but inner branches do little)

Exact stream bookkeeping differs by mode (String / DOM / Stream); see §4.

4. Runtime algorithm by storage mode

All paths wrap unexpected failures as:

```
EIPException("Error in SaveDataToAttributeProcessor.", cause)
```

Missing attributes use:

- Local: Couldn't load attribute {name} - no attribute found.
- Global: Couldn't load global attribute '{name}' in bucket '{bucket}' - no attribute found.

4.1 DOM mode (USEDOCUMENTOBJECTMODEL = true)

Checked **first** in processData.

Data -> Attribute (and optional swap)

1. Copy current data stream into an **EIP Cache**.
2. Borrow namespace-aware **DocumentBuilder** from module pool; **parse** cache input -> **Document inputDoc**.
3. If also Attribute -> Data (swap):
 - Read existing attribute as **Document** (global or local); null/NPE -> error.
 - Set attribute to **inputDoc**.
 - Set data stream to XML string of the **previous** document element (**DOMUtil.buildStringFromElement**).
4. Else (save only):
 - Set attribute (global or local) to **inputDoc**.
 - Restore data stream from **cache.getInputStream()**.
 - Return builder to pool (swap branch does not show the same return in all paths – pool discipline differs by branch in source; treat DOM builder pooling as implementation-sensitive).

Attribute -> Data only

1. Load **Document** from attribute (must exist).
2. **DOMUtil.saveDocument** into a new cache; set data stream to that cache input.

Implication: Attribute -> Data in DOM mode replaces the transaction stream with serialized XML of the stored **Document**.

4.2 Stream mode (**StoreAsStream = true**, **DOM false**)

Data -> Attribute

1. Size cache from **data.getDataStream().available()** (note: **available()** is not always full length for all stream types).
2. Copy stream into cache; store **cache.getInputStream()** on the attribute (global or local).
3. If swap (Attribute -> Data also true):
 - Re-read attribute object; must be non-null **InputStream**.
 - **Important quirk:** after save, the attribute now holds the **new** stream. The swap branch then sets data from that same attribute read – i.e. this is not a classic “exchange with prior value” in the String/DOM sense; review carefully before relying on dual-true in Stream mode.
4. Else restore **data.setDataStream(cache.getInputStream())**.

Attribute -> Data only

1. Load attribute; null -> error.
2. Cast to **InputStream** and set as data stream.

4.3 Plain String mode (neither DOM nor Stream)

Data -> Attribute

1. Copy current stream into cache; read bytes to **String inbound**.
2. Reset data stream to **cache.getInputStream()** (rewind of the copy).
3. If swap:

- Load existing attribute as **String**; null -> error.
 - Write **inbound** into attribute (global/local).
 - Set data stream to **StringInputStream(previousAttributeString)**.
4. Else save only: write **inbound** into attribute; data stream already reset to cache copy.

Attribute -> Data only

1. Load attribute **String**; null -> error.
2. Best-effort **data.getDataStream().close()** (errors logged, not fatal).
3. **data.setDataStream(new StringInputStream(attributeData))**.

Implication: Plain mode materializes the full payload as a Java **String** (charset = platform default via **new String(bytes)**), which can be costly or lossy for binary data. Prefer **Use Stream** for binaries.

5. Decision tree (simplified)

```
processData(data)
  read flags: storeAsStream, isDom, dataToAttribute, attributeToData, isGlobal, bucket, name
  if isDom:
    DOM save / load / swap paths
  else if storeAsStream:
    Stream save / load / swap paths
  else:
    String save / load / swap paths
  return data
```

If both DOM and Stream are enabled in configuration, **DOM wins** because it is the first branch.

6. Inherited AbstractProcessor options

getConfigurationDescriptor() starts with **super.getConfigurationDescriptor()**.

From decompiled **AbstractProcessor** in this WAR:

UI (i18n key)	Tag	Default	Tab
Conditional execution label (AbstractProcessor.ConditionalExecution.label)	ExecuteProcess	none	CONDITIONAL_EXECUTION

Tooltip comes from **AbstractProcessor.ConditionalExecution.tooltip** (localized). When disabled via conditional execution, the processor is skipped by the framework (**isExecute**).

This module does not call **registerTransactionAttribute**; it **reads/writes** the configured attribute key at runtime (and optional global bucket).

7. Worked examples

7.1 Park payload, run a side path, restore (two stages)

Stage A – save

- Copy Data to Attribute = true
- Copy Attribute to Data = false
- Attribute Name = `com.acme.OriginalPayload`
- Use Stream = true (if binary) or plain String (if small text)

Later Stage B – restore

- Copy Data to Attribute = false
- Copy Attribute to Data = true
- Same attribute name / mode

7.2 Swap for XSLT that needs alternate document

- Both copy directions true
- Use Document Object Model = true
- Attribute already holds a **Document** from a prior stage
- After processor: stream is previous DOM serialized; attribute holds newly parsed current stream

7.3 Share a value across routes (global)

- Global Attribute = true
 - Global Attribute Bucket = **SHARED**
 - Copy Data to Attribute = true
 - Later transactions use Copy Attribute to Data against the same bucket+name
-

8. Known quirks and caveats

1. **DOM vs Stream mutual exclusion** is documented in tooltips; code does not throw if both true – DOM path runs first.
2. **Missing attributes always error** when loading (Copy Attribute to Data), including swap cases that need the prior value.
3. **Plain String mode** uses `new String(bytes)` without an explicit charset – beware non-UTF-8 payloads.
4. **Stream mode + dual direction** does not mirror the classic String/DOM “exchange with previous value” as cleanly; validate before using swap with streams.
5. **Global attributes are in-memory process state** – not durable across JVM restart.
6. **available() sizing** in Stream save may under-allocate for some stream types; EIP cache still accepts the copy, but be aware of the hint.
7. **DocumentBuilder pool**: DOM parse uses a shared pool with namespace awareness forced true for this call.
8. Class name is historical (`SaveDataToAttributeProcessor`); UI type string is **Data Attribute Swapper**.

9. Operational recommendations

Goal	Settings
Save text for later in same transaction	Data->Attr true; Attr->Data false; plain mode
Large or binary payloads	Use Stream
Expose XML to XSLT as DOM attribute	Use Document Object Model
True exchange of two XML docs	Both directions; DOM mode; ensure attribute pre-exists
Cross-transaction coordination	Global Attribute + dedicated bucket name
Avoid collisions	Explicit Attribute Name; never leave the default unless intentional

10. Error messages

Condition	Message
Missing local attribute on load/swap	Couldn't load attribute {name} - no attribute found.
Missing global attribute on load/swap	Couldn't load global attribute '{name}' in bucket '{bucket}' - no attribute found.
Any other failure	Error in SaveDataToAttributeProcessor. (wrapped cause)

11. Source index (this WAR)

Topic	Path under war-pipeline/decompiled/eip-26R1.11/
Data Attribute Swapper	com/pilotfish/eip/modules/internal/SaveDataToAttr
AbstractProcessor	com/pilotfish/eip/extend/AbstractProcessor.java
TransactionAttributes	com/pilotfish/eip/TransactionAttributes.java
Module JAR	war-pipeline/jars/eip-26R1.11/xcs-modules-core-1.0

12. Pipeline provenance

PilotFish WARs/eip.war.hs.26R1.11

-> extract/eip-26R1.11/

-> jars: xcs-modules-core-1.0.3.jar (+ xcs-core)

-> decompiled/eip-26R1.11/ (CFR 0.152)

-> this deep-dive Markdown/PDF

Deep-dive documentation from WAR decompile – Data Attribute Swapper – 26R1.11 – July 2026.